

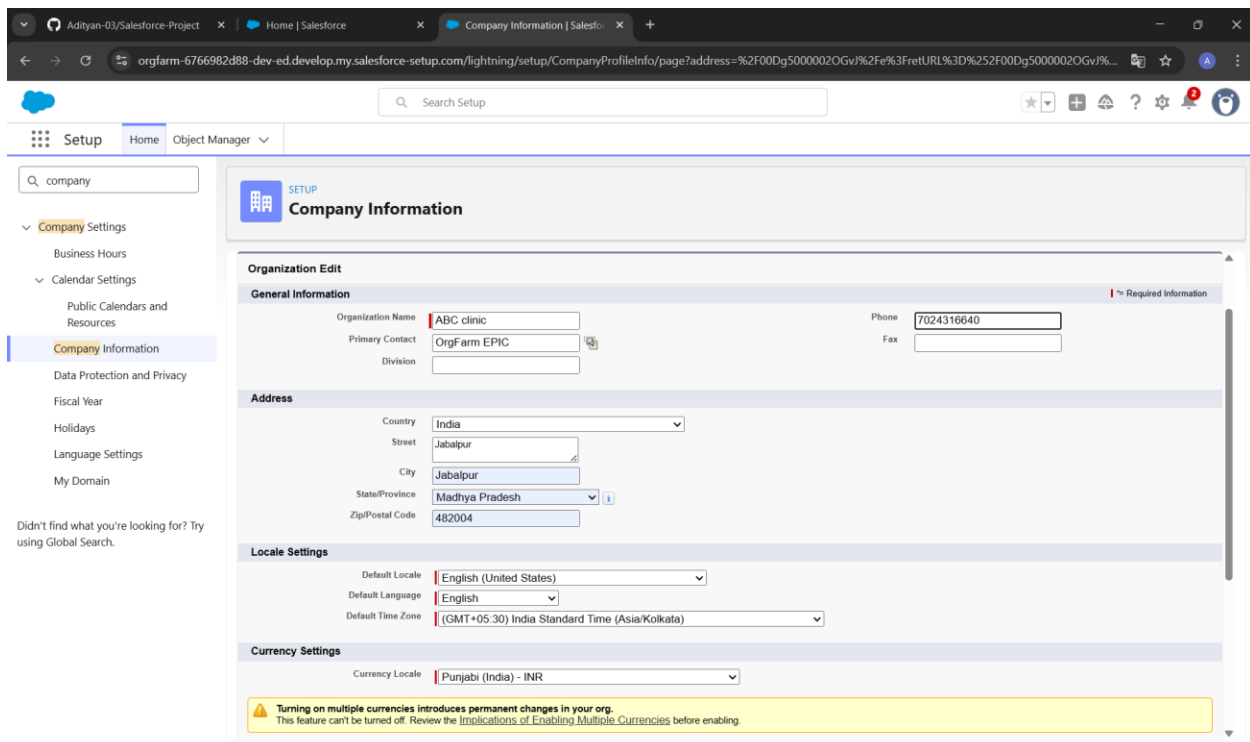
# Clinic Appointment & Patient Management CRM

## Final Project Documentation

### Project Overview

Small and medium-sized clinics often struggle with manual appointment scheduling, patient record management, and follow-ups. This project implements a Salesforce-based CRM solution to centralize patient data, automate appointment workflows, and provide actionable insights through reports and dashboards. The system improves operational efficiency, reduces missed appointments, and enhances patient satisfaction.

### Objectives



The screenshot displays the Salesforce Setup interface for editing company information. The left sidebar shows the navigation menu with 'Company Information' selected. The main content area is titled 'Organization Edit' and contains several sections: 'General Information' with fields for Organization Name (ABC clinic), Primary Contact (OrgFarm EPIC), Division, Phone (7024316640), and Fax; 'Address' with fields for Country (India), Street (Jabalpur), City (Jabalpur), State/Province (Madhya Pradesh), and Zip/Postal Code (482004); 'Locale Settings' with fields for Default Locale (English (United States)), Default Language (English), and Default Time Zone ((GMT+05:30) India Standard Time (Asia/Kolkata)); and 'Currency Settings' with a field for Currency Locale (Punjabi (India) - INR). A yellow warning banner at the bottom states: 'Turning on multiple currencies introduces permanent changes in your org. This feature can't be turned off. Review the Implications of Enabling Multiple Currencies before enabling.'

- Digitally manage patient and doctor records
- Automate appointment scheduling and reminders

- Prevent data errors and double bookings
  - Provide real-time visibility using reports and dashboards
  - Improve clinic productivity and patient engagement
- 

## **Phase 1: Problem Understanding & Industry Analysis**

### **Problem Statement**

Clinics rely on manual registers, spreadsheets, and phone calls to manage appointments and patient records. This leads to missed appointments, data loss, double booking of doctors, and lack of visibility into clinic operations. Doctors do not have easy access to patient history, and administrators lack real-time reports.

The proposed solution is a Salesforce-based Clinic Appointment & Patient Management CRM to centralize data, automate processes, and provide analytics.

### **Requirement Gathering – Key Problems**

1. Manual patient record management
2. Inefficient appointment scheduling
3. No automated reminders
4. Poor visibility for doctors
5. Limited reporting and analytics

### **Stakeholder Analysis**

- Clinic Admin: Needs dashboards and overall control
- Receptionist: Needs fast booking and record management
- Doctors: Need daily schedules and patient history
- Patients: Need confirmations and reminders
- System Admin: Needs secure and stable CRM

### **Business Process Mapping**

#### **As-Is Process**

- Manual patient entry

- Phone-based booking
- No reminders
- Paper-based history
- Manual reporting

#### **To-Be Process**

- Centralized Salesforce records
  - Automated booking and status tracking
  - Automated email reminders
  - Easy access to patient history
  - Real-time dashboards
- 

### **Phase 2: Org Setup & Configuration**

#### **Configurations Completed**

- Salesforce Developer Org created
- Company Profile set as ABC Clinic
- Business Hours: Monday–Saturday, 9 AM to 6 PM
- Holidays configured
- Standard Fiscal Year configured

#### **User Setup**

- Admin User
- Receptionist User
- Doctor User

#### **Security & Access**

- Profiles assigned based on roles
- Login access policies reviewed
- Deployment basics and sandbox concept documented

**Outcome:** Salesforce organization prepared to support clinic operations.

---

### **Phase 3: Data Modeling & Relationships**

#### **Custom Objects Created**

- Patient
- Doctor
- Appointment

#### **Fields**

**Patient:** Phone, Email, Age, Gender, Address

**Doctor:** Specialization, Phone, Email, Availability

**Appointment:** Appointment Date, Time, Status, Reason

#### **Relationships**

- Patient → Appointment (Master-Detail)
- Doctor → Appointment (Lookup)

#### **UI Configuration**

- Page Layouts customized
- Compact Layouts created
- Schema Builder used to visualize relationships

**Outcome:** Structured data model implemented with proper relationships.

---

### **Phase 4: Process Automation (Admin)**

#### **Automations Implemented**

1. **Validation Rule**
  - Prevent appointment booking on past dates
2. **Record-Triggered Flow**
  - Automatically set Status = Scheduled on creation
3. **Email Alert**

- Send appointment confirmation email

#### 4. **Scheduled Flow**

- Send reminder email one day before appointment

#### 5. **Task Automation**

- Create follow-up task when appointment is completed

**Outcome:** Manual work reduced and appointment management automated.

---

### **Phase 5: Apex Programming (Skipped)**

Phase 5 was skipped as the project focuses mainly on Admin and UI-driven implementation. Complex logic is handled using declarative automation wherever possible.

---

### **Phase 6: User Interface Development**

#### **UI Components Built**

- Custom Lightning App: Clinic CRM
- Custom Tabs: Patient, Doctor, Appointment
- Navigation Bar configured

#### **Record Pages**

- Patient Record Page
- Doctor Record Page
- Appointment Record Page

#### **Home Page**

- Clinic Home with reports and recent items

#### **Utility Bar**

- Notes
- Recent Items

**Outcome:** User-friendly interface built for clinic staff and doctors.

---

## **Phase 7: Integration & External Access**

### **Configurations**

- Remote Site Settings configured
- Named Credentials created
- OAuth via Connected App explained
- API Limits reviewed

### **Concepts Covered**

- REST API exposure
- Callouts for SMS/Email services
- External Services
- Platform Events and Change Data Capture (conceptual)

**Outcome:** System prepared for secure future integrations.

---

## **Phase 8: Data Management & Deployment**

### **Data Management**

- Data Import Wizard used for patient data
- Duplicate Rules configured for Email and Phone
- Data Export & Backup explained

### **Deployment**

- Change Sets explained
- Managed vs Unmanaged Packages documented
- SFDX and VS Code deployment concepts covered

**Outcome:** Data quality ensured and deployment strategy defined.

---

## **Phase 9: Reports, Dashboards & Security**

## **Reports**

- Appointments by Date
- Appointments by Doctor
- Missed Appointments

## **Dashboards**

- Today's Appointments
- Total Patients
- Appointment Status Summary

## **Security**

- Profiles and Roles configured
- Permission Sets assigned
- OWD and Sharing Rules explained
- Field Level Security applied

**Outcome:** Strong analytics and secure access control implemented.

---

## **Phase 10: Quality Assurance Testing**

### **Testing Approach**

- Test cases prepared for:
  - Patient creation
  - Appointment booking
  - Automation flows
  - Email alerts

### **Test Case Format**

- Scenario
- Test Steps
- Expected Result

- Actual Result with screenshots

**Outcome:** All features tested and validated successfully.

---

## **Conclusion**

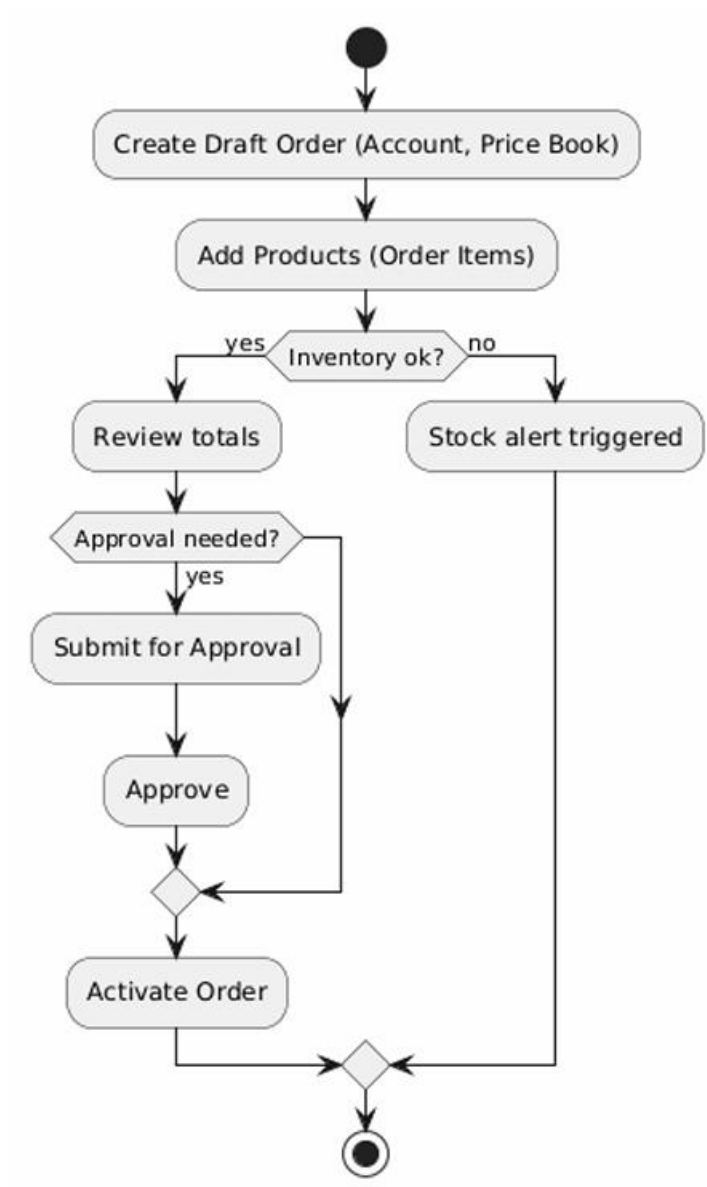
The Clinic Appointment & Patient Management CRM successfully automates clinic operations by centralizing patient data, automating appointments and reminders, and providing real-time insights. The system reduces manual effort, improves patient experience, and enhances decision-making. Future enhancements may include SMS integration, payment gateway integration, and AI-based scheduling.

---

## **Future Enhancements**

- SMS and WhatsApp integration
  - Online patient portal
  - AI-based appointment prediction
  - Payment integration
  - Mobile app support
-





# Phase 2: Org Setup & Configuration

---

## 2.1 Salesforce Org Provisioning

A Salesforce Developer Edition organization was created to serve as the primary development and configuration environment for this project. The Developer Org provides a cloud-based platform where all objects, automations, and UI components are designed and tested.

Key activities performed:

- Registration and activation of Salesforce Developer Org.
- Initial login and verification of system access.
- Familiarization with Lightning Experience interface and Setup menu.

Business Value: The Developer Org enables isolated development and testing without impacting any production environment. It supports rapid prototyping and experimentation.

---

## 2.2 Company Profile Configuration

The Company Profile was configured to reflect the clinic's organizational identity and regional settings. This ensures that all system-generated records, emails, and reports follow the clinic's operational context.

Configurations performed:

- Company Name set as: ABC Clinic
- Address updated with clinic location
- Default Language set to English
- Time Zone set to India Standard Time (IST)

Business Value: Correct company profile configuration ensures accurate time-based automations, reporting consistency, and proper localization of system behavior.

The screenshot shows the Salesforce Setup interface for 'Company Information'. The left sidebar contains a search bar with 'company' and a list of settings: Company Settings, Business Hours, Calendar Settings, Public Calendars and Resources, Company Information (highlighted), Data Protection and Privacy, Fiscal Year, Holidays, Language Settings, and My Domain. Below this is a message: 'Didn't find what you're looking for? Try using Global Search.' The main content area is titled 'Company Information' and contains the 'Organization Edit' form. The form is divided into sections: General Information, Address, Locale Settings, and Currency Settings. The General Information section includes fields for Organization Name (ABC clinic), Primary Contact (OrgFarm EPIC), Division, Phone (7024316640), and Fax. The Address section includes fields for Country (India), Street (Jabalpur), City (Jabalpur), State/Province (Madhya Pradesh), and Zip/Postal Code (482004). The Locale Settings section includes fields for Default Locale (English (United States)), Default Language (English), and Default Time Zone ((GMT+05:30) India Standard Time (Asia/Kolkata)). The Currency Settings section includes a field for Currency Locale (Punjabi (India) - INR). A yellow warning banner at the bottom states: 'Turning on multiple currencies introduces permanent changes in your org. This feature can't be turned off. Review the Implications of Enabling Multiple Currencies before enabling.'

Adityan-03/Salesforce Project x Home | Salesforce x Company Information | Salesforce x

orgfarm-6766982d88-dev-ed.develop.my.salesforce-setup.com/lightning/setup/CompanyProfileInfo/page?address=%2F00Dg5000002OGvJ%2Fe%3FretURL%3D%252F00Dg5000002OGvJ%...

Search Setup

Setup Home Object Manager

company

Company Settings

Business Hours

Calendar Settings

Public Calendars and Resources

Company Information

Data Protection and Privacy

Fiscal Year

Holidays

Language Settings

My Domain

Didn't find what you're looking for? Try using Global Search.

SETUP

Company Information

Organization Edit

General Information

Organization Name ABC clinic

Primary Contact OrgFarm EPIC

Division

Phone 7024316640

Fax

Address

Country India

Street Jabalpur

City Jabalpur

State/Province Madhya Pradesh

Zip/Postal Code 482004

Locale Settings

Default Locale English (United States)

Default Language English

Default Time Zone (GMT+05:30) India Standard Time (Asia/Kolkata)

Currency Settings

Currency Locale Punjabi (India) - INR

Turning on multiple currencies introduces permanent changes in your org. This feature can't be turned off. Review the Implications of Enabling Multiple Currencies before enabling.

## 2.3 Business Hours Setup

Clinic working hours were configured using Salesforce Business Hours to control appointment scheduling, reminders, and service availability.

Configured Schedule:

- Monday to Saturday: 9:00 AM to 6:00 PM
- Sunday: Closed

Business Value: Business hours restrict system operations to valid working times, preventing appointment scheduling outside clinic hours and ensuring accurate reminder delivery.

The screenshot shows the Salesforce Setup interface for configuring Organization Business Hours. The left sidebar contains a navigation menu with 'Company Settings' expanded, showing 'Business Hours' as the selected option. The main content area is titled 'Organization Business Hours' and includes a description: 'Select the days and hours that your support team is available. These hours, when associated with escalation rules, determine the times at which cases can escalate. If you enter blank business hours for a day, that means your organization does not operate on that day.' Below this is the 'Business Hours Edit' form. The form has three steps: Step 1 'Business Hours Name' with a text field containing 'Default' and a checkbox 'Active' checked; Step 2 'Time Zone' with a dropdown menu set to '(GMT+05:30) India Standard Time (Asia/Kolkata)'; and Step 3 'Business Hours' which is a table for configuring hours for each day of the week. The table has columns for the day, start time, end time, and a checkbox for '24 hours'. The current configuration shows hours from 9:00 AM to 6:00 PM for all days, with the '24 hours' checkbox unchecked for all days. The form has 'Save' and 'Cancel' buttons at the bottom.

Setup | Home | Object Manager

Search Setup

company

Company Settings

- Business Hours
- Calendar Settings
  - Public Calendars and Resources
- Company Information
  - Data Protection and Privacy
  - Fiscal Year
  - Holidays
  - Language Settings
  - My Domain

Didn't find what you're looking for? Try using Global Search.

### Organization Business Hours

Select the days and hours that your support team is available. These hours, when associated with escalation rules, determine the times at which cases can escalate. If you enter blank business hours for a day, that means your organization does not operate on that day.

**Business Hours Edit** [Save] [Cancel]

**Step 1. Business Hours Name** [Required Information]

Business Hours Name:  Use these business hours as the default: ☒

Active: ☒

**Step 2. Time Zone**

Time Zone:

**Step 3. Business Hours**

| Day       | Start Time                           | End Time                             | 24 hours                 |
|-----------|--------------------------------------|--------------------------------------|--------------------------|
| Sunday    | <input type="text" value="9:00 AM"/> | <input type="text" value="6:00 PM"/> | <input type="checkbox"/> |
| Monday    | <input type="text" value="9:00 AM"/> | <input type="text" value="6:00 PM"/> | <input type="checkbox"/> |
| Tuesday   | <input type="text" value="9:00 AM"/> | <input type="text" value="6:00 PM"/> | <input type="checkbox"/> |
| Wednesday | <input type="text" value="9:00 AM"/> | <input type="text" value="6:00 PM"/> | <input type="checkbox"/> |
| Thursday  | <input type="text" value="9:00 AM"/> | <input type="text" value="6:00 PM"/> | <input type="checkbox"/> |
| Friday    | <input type="text" value="9:00 AM"/> | <input type="text" value="6:00 PM"/> | <input type="checkbox"/> |
| Saturday  | <input type="text" value="HH:MM"/>   | <input type="text" value="HH:MM"/>   | <input type="checkbox"/> |

[Save] [Cancel]

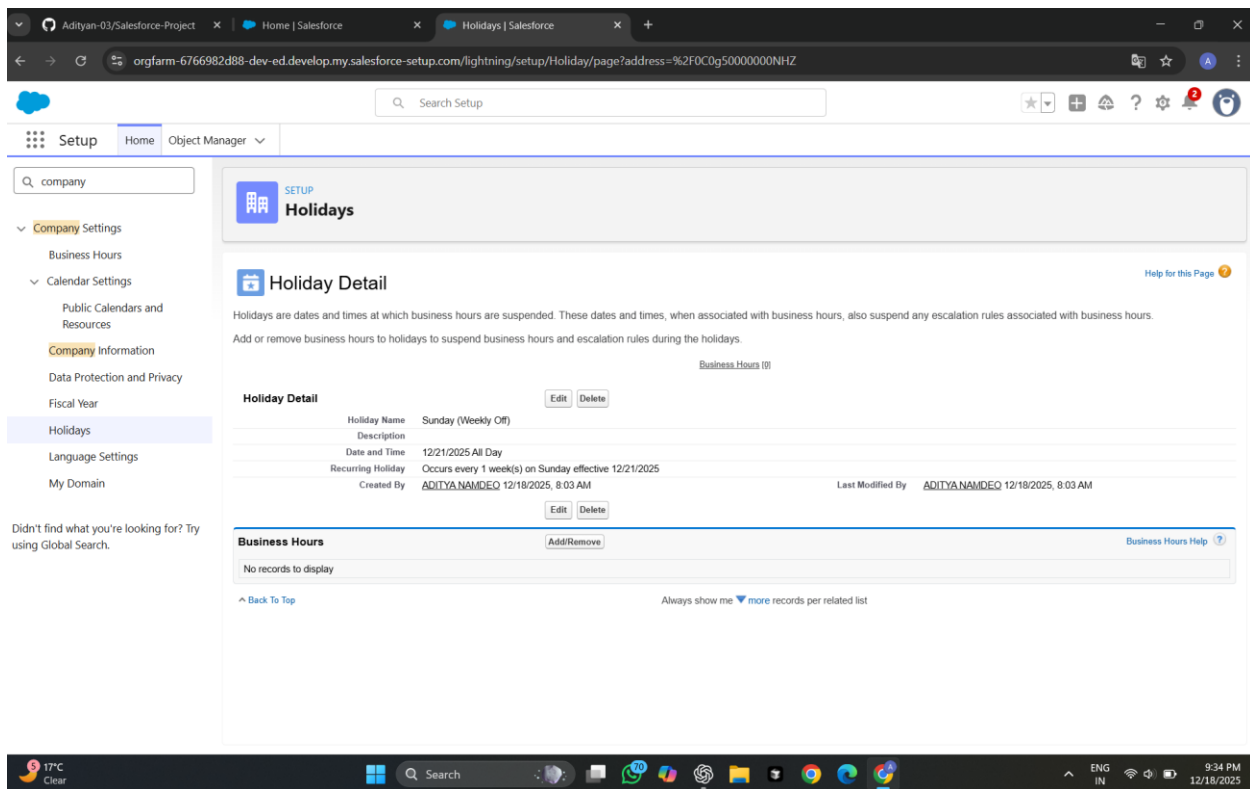
## 2.4 Holiday Configuration

Clinic holidays and weekly offs were configured using the Holiday calendar.

Configured Holidays:

- Weekly off: Sunday
- Optional national holidays

Business Value: Holiday configuration prevents accidental appointment booking on non-working days and ensures correct scheduling logic for reminders and workflows.



## 2.5 Fiscal Year Settings

The fiscal year was configured to align with Indian financial reporting standards.

Configuration:

- Fiscal Year Type: Standard
- Start Month: April

Business Value: Correct fiscal year settings ensure accurate financial and operational reporting, especially when generating yearly performance dashboards and summaries.

## 2.6 User Management and Role Definition

Multiple users were created to represent real clinic roles. Each user was assigned an appropriate profile to enforce role-based access.

Users Created:

- System Administrator (Full access)

- Receptionist (Operational access)
- Doctor (Read and limited edit access)

Business Value: User-based configuration ensures accountability, auditability, and controlled access to sensitive patient and appointment data.

## 2.7 Profile Assignment and Access Control

Profiles were assigned to each user based on job responsibility.

Profile Mapping:

- Admin User → System Administrator Profile
- Receptionist → Standard User Profile
- Doctor → Standard User Profile (with limited permissions)

Business Value: Profiles control what objects, fields, and actions a user can access, ensuring data security and compliance with healthcare data protection principles.

The screenshot displays the Salesforce Setup page for Users. The left sidebar shows the navigation menu with 'Users' selected. The main content area is titled 'All Users' and includes a table of users. The table has columns for Action, Full Name, Alias, Username, Role, Active, and Profile. The users listed are:

| Action | Full Name        | Alias   | Username                                                    | Role | Active | Profile                          |
|--------|------------------|---------|-------------------------------------------------------------|------|--------|----------------------------------|
| [Edit] | 1.ADMIN          | a1      | admin.121@gmail.com                                         |      | ✓      | Partner App Subscription User    |
| [Edit] | 2.Receptionist   | r2      | receptionist.1@gmail.com                                    |      | ✓      | Partner App Subscription User    |
| [Edit] | 3.Doctor         | d3      | doctor.123@gmail.com                                        |      | ✓      | Einstein Agent User              |
| [Edit] | Chatter Expert   | Chatter | chatty.00d9500000209yeaw.mze25ij0u8o@chatter.salesforce.com |      | ✓      | Chatter Free User                |
| [Edit] | EPIC_OrgFarm     | OEPIG   | epic.026f5d62ee3@orgfarm.salesforce.com                     |      | ✓      | System Administrator             |
| [Edit] | NAMDEO ADITYA    | adi     | adityanamdeo3107594@agentforce.com                          |      | ✓      | System Administrator             |
| [Edit] | User_Integration | integ   | integration@00d9500000209yeaw.com                           |      | ✓      | Analytics Cloud Integration User |
| [Edit] | User_Security    | sec     | insightssecurity@00d9500000209yeaw.com                      |      | ✓      | Analytics Cloud Security User    |

## **2.8 Login Access Policies and Session Security**

Basic login and session security settings were reviewed and configured to protect clinic data from unauthorized access.

Configurations:

- Enforced password policies
- Reviewed login IP relaxation settings
- Session timeout policies verified

Business Value: Strong authentication and session management protect sensitive patient information from misuse and unauthorized access.

---

## **2.9 Development Environment and Sandbox Concept**

The concepts of development lifecycle and sandbox usage were documented.

Key Points:

- Developer Org used for configuration and testing.
- Sandbox recommended for testing future enhancements before production deployment.

Business Value: Environment separation reduces deployment risks and ensures system stability.

---

## **2.10 Deployment Basics and Change Management**

Basic deployment concepts were introduced to support controlled movement of configurations.

Covered Topics:

- Change Sets for metadata deployment
- Difference between Development and Production environments
- Importance of version control

Business Value: Deployment management ensures safe and traceable promotion of changes across environments.

---

# Phase 3: Data Modeling & Relationships

## Purpose of Phase 3

The purpose of Phase 3 is to design and implement a robust data architecture for the Clinic Appointment & Patient Management CRM. This phase defines how clinic data is structured, stored, related, and accessed within Salesforce. A well-designed data model ensures data integrity, scalability, performance, and ease of reporting.

---

### 3.1 Data Model Planning and Object Identification

The first activity in this phase was to identify the core business entities involved in clinic operations. Based on requirement analysis, three primary entities were identified:

- Patient: Represents individuals visiting the clinic.
- Doctor: Represents medical practitioners providing consultation.
- Appointment: Represents the interaction between a patient and a doctor at a scheduled time.

These entities were mapped to Salesforce Custom Objects to allow full control over structure and behavior.

Business Value: Clear object identification ensures that each business concept is stored independently and can scale as clinic operations grow.

---

### 3.2 Creation of Custom Objects

Three custom objects were created using Salesforce Object Manager.

#### Patient Object

- Label: Patient
- Plural Label: Patients
- Record Name: Patient Name (Text)
- Reports and Activities enabled

Purpose: The Patient object stores demographic and contact information of all clinic visitors.



## Doctor Object

- Label: Doctor
- Plural Label: Doctors
- Record Name: Doctor Name (Text)
- Reports and Activities enabled

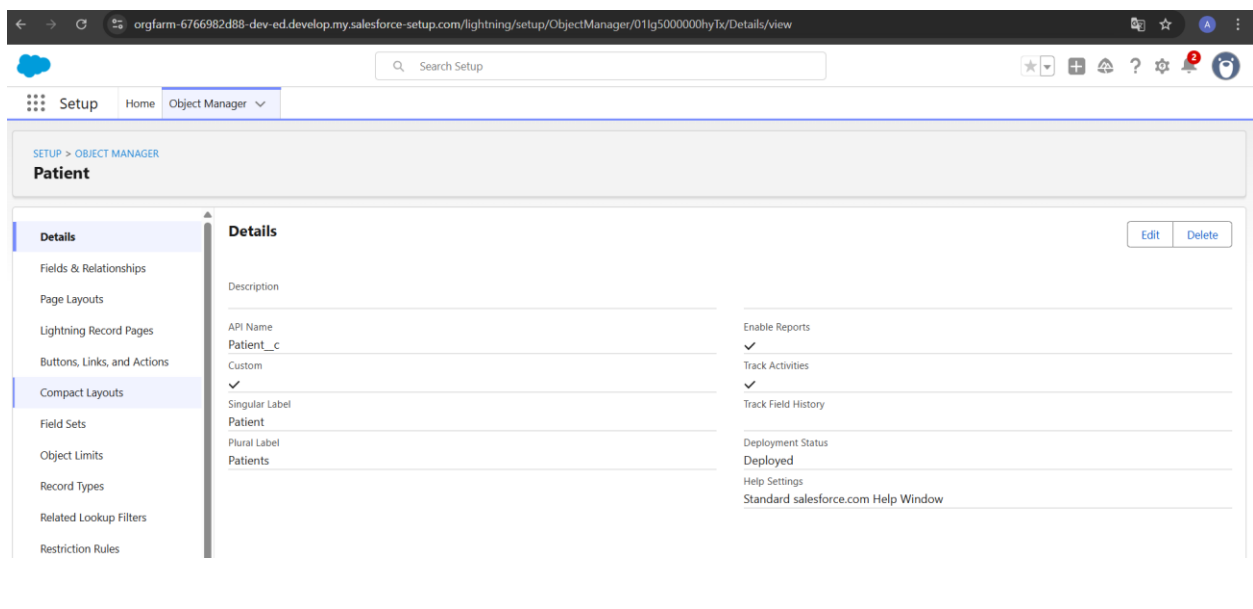
Purpose: The Doctor object stores doctor profiles, specialization, and availability details.

## Appointment Object

- Label: Appointment
- Plural Label: Appointments
- Record Name: Appointment Number (Auto Number)
- Reports and Activities enabled

Purpose: The Appointment object manages booking details and acts as the transactional object connecting patients and doctors.

Business Value: Custom objects allow full customization aligned with clinic-specific workflows and data needs.



## 3.3 Field Design and Data Type Selection

For each object, essential fields were created with appropriate data types to ensure accurate data capture and validation.

## Patient Object Fields

- Phone Number (Phone)
- Email (Email)

The screenshot shows the Salesforce Setup interface for the 'Patient' object. The left sidebar contains a navigation menu with options like Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Restriction Rules, Scoping Rules, Object Access, Triggers, Flow Triggers, and Validation Rules. The main content area is titled 'Fields & Relationships' and displays a table of 9 items, sorted by Field Label. The table has columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed. The fields listed are Address, Age, Created By, Email, Gender, Last Modified By, Owner, Patient Name, and Phone Number.

| FIELD LABEL      | FIELD NAME       | DATA TYPE           | CONTROLLING FIELD | INDEXED |
|------------------|------------------|---------------------|-------------------|---------|
| Address          | Address__c       | Text Area(255)      |                   |         |
| Age              | Age__c           | Number(18, 0)       |                   |         |
| Created By       | CreatedById      | Lookup(User)        |                   |         |
| Email            | Email__c         | Email               |                   |         |
| Gender           | Gender__c        | Picklist            |                   |         |
| Last Modified By | LastModifiedById | Lookup(User)        |                   |         |
| Owner            | OwnerId          | Lookup(User, Group) |                   | ✓       |
| Patient Name     | Name             | Text(80)            |                   | ✓       |
| Phone Number     | Phone_Number__c  | Phone               |                   |         |

# Phase 4: Process Automation (Admin)

## 4.1 Automation Strategy and Tool Selection

Based on the business requirements identified in Phase 1, the following automation tools were selected:

- **Validation Rules** – To enforce data integrity and prevent incorrect data entry.
- **Record-Triggered Flows** – To automate actions at the time of record creation or update.
- **Scheduled Flows** – To perform time-based automation such as reminders.
- **Email Alerts** – To automate patient communication.
- **Task Automation** – To ensure follow-up activities are not missed.

## 4.2 Validation Rule – Prevent Past Appointment Booking

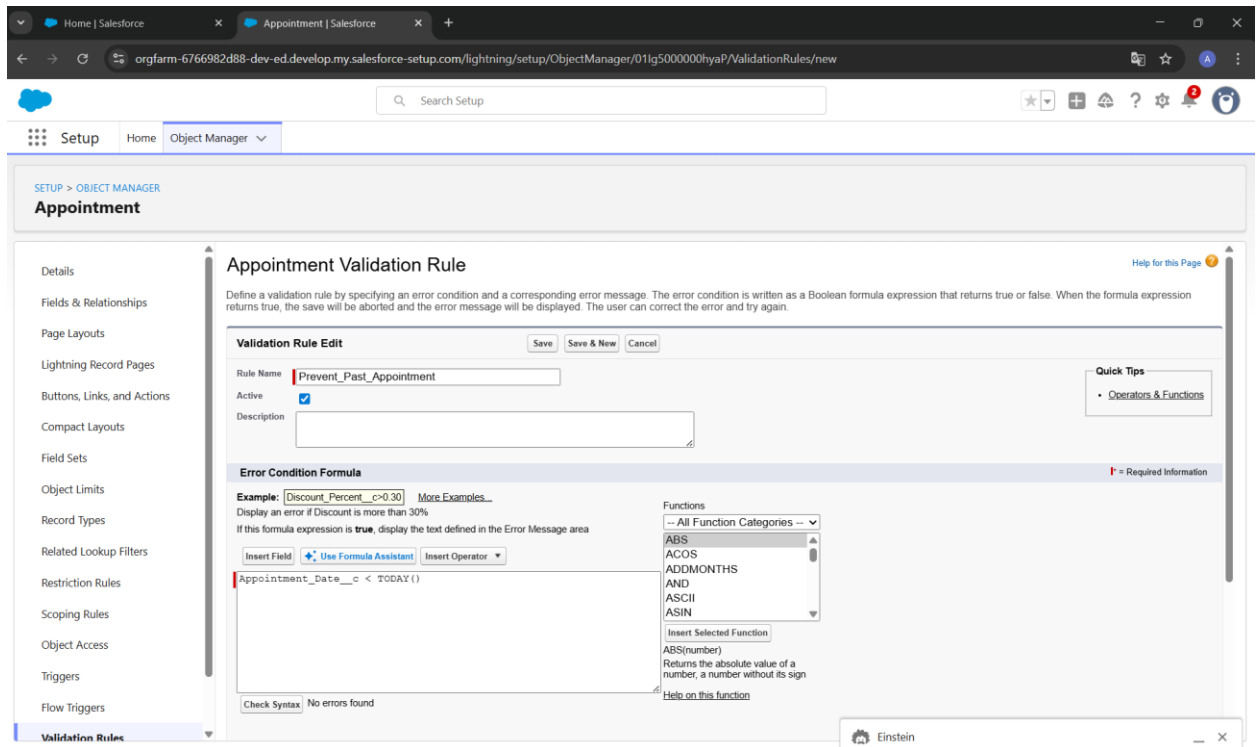
### Business Requirement

Appointments should not be scheduled for past dates. Allowing past dates would create invalid records, reporting inaccuracies, and operational confusion.

### Implementation

A validation rule was created on the **Appointment** object with the following logic:

- **Condition:**  
Appointment Date < TODAY()
- **Error Message:**  
*"Appointment date cannot be in the past."*
- **Error Location:**  
Displayed at the Appointment Date field.



## 4.3 Record-Triggered Flow – Auto Set Default Appointment Status

### Business Requirement

When a new appointment is created, its status should automatically be set to **“Scheduled”** to maintain consistency and avoid manual updates.

### Implementation

A Record-Triggered Flow was created on the **Appointment** object with the following configuration:

- **Trigger:** When a record is created
- **Entry Condition:** None (runs for all new appointments)
- **Action:** Update the field Status = Scheduled

This flow runs immediately after record creation.

## 4.4 Email Alert – Appointment Confirmation Notification

### Business Requirement

Patients should receive an automatic confirmation email immediately after an appointment is booked.

## Implementation

The following components were created:

### 1. Email Template

- Contains appointment details such as date and patient name.

### 2. Email Alert

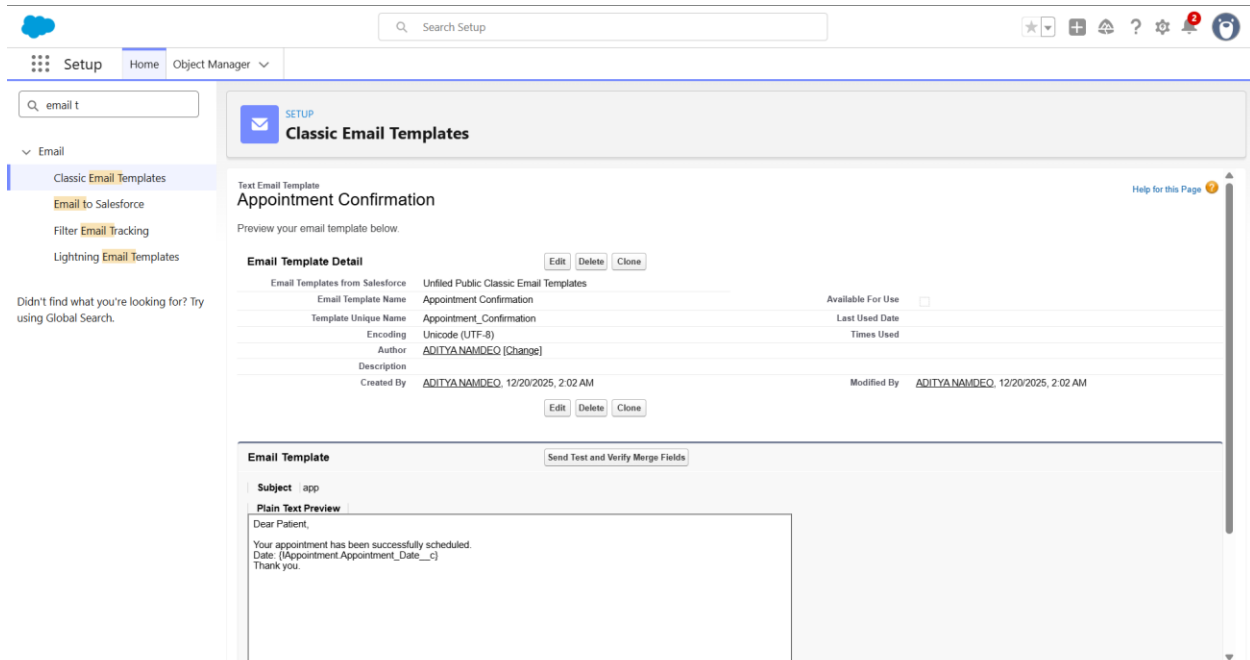
- Object: Appointment
- Recipient: Patient Email
- Template: Appointment Confirmation

The screenshot shows the Salesforce Setup interface for Email Alerts. The left sidebar contains a search bar with "email a" and a list of navigation items: Email, Email Address, Internationalization, Email Attachments, Process Automation, Workflow Actions, and Email Alerts (highlighted). The main content area is titled "Email Alerts" and shows a list of alerts. The first alert, "done", is selected and its details are displayed. The details include: Description (done), Unique Name (do1ne), From Email Address (Current User's email address), Recipients (User: Receptionist 2), Additional Emails, Created By (ADITYA NAMDEO, 12/20/2025, 2:05 AM), Modified By (ADITYA NAMDEO, 12/20/2025, 2:05 AM), Email Template (Canceled Service Appointment Confirmation Email), and Object (Appointment). Below the details are sections for "Rules Using This Email Alert", "Approval Processes Using This Email Alert", "Entitlement Processes Using This Email Alert", and "Flows Using This Email Alert". The "Rules Using This Email Alert" section shows that the alert is currently not used by any rules. The "Approval Processes Using This Email Alert" and "Entitlement Processes Using This Email Alert" sections also show that the alert is currently not used by any approval or entitlement processes. The "Flows Using This Email Alert" section shows a table with columns for Flow Name, Version, Description, Object, and Active.

| Flow Name | Version | Description | Object | Active |
|-----------|---------|-------------|--------|--------|
|-----------|---------|-------------|--------|--------|

### 3. Triggering Mechanism

- Email alert is invoked from the Record-Triggered Flow on appointment creation.



## 4.5 Scheduled Flow – Appointment Reminder Automation

### Business Requirement

Patients often forget appointments. A reminder should be sent automatically **one day before** the scheduled appointment date.

### Implementation

A Scheduled-Triggered Flow was created with the following configuration:

- **Schedule:** Daily execution
- **Object:** Appointment
- **Condition:**
  - Appointment Date = TODAY() + 1
  - Status = Scheduled

# Phase 5: Apex Programming (Developer)

## 5.1 Apex Programming Strategy

The Apex implementation strategy follows these principles:

- Use Apex only where declarative tools are insufficient
- Follow clean coding and trigger best practices
- Separate business logic from trigger logic
- Ensure test coverage and system stability

### Apex Components Implemented

- Apex Trigger – Prevent double booking
- Apex Handler Class – Encapsulate business logic
- SOQL Queries – Fetch and filter appointment records
- Collections – Store and process multiple records
- Scheduled Apex – Auto-update missed appointments
- Test Classes – Ensure minimum 75% code coverage

## 5.2 Apex Trigger – Prevent Double Booking of Doctors

### Business Requirement

A doctor should not be assigned more than one appointment at the same date and time. Double booking leads to operational conflicts, patient dissatisfaction, and scheduling inefficiencies.

Declarative validation is insufficient for this scenario because it requires comparing the current record with multiple existing records in the database.

### Implementation

An Apex Trigger was created on the **Appointment** object with the following characteristics:

- Trigger Events: Before Insert, Before Update

- Logic:
  - Collect doctor IDs from incoming appointments
  - Query existing appointments for those doctors
  - Compare date and time values
  - If a conflict is found, prevent save using addError()

```
trigger AppointmentTrigger on Appointment__c (before insert, before update) {  
  
    // Call handler class method  
    if (Trigger.isBefore && (Trigger.isInsert || Trigger.isUpdate)) {  
        AppointmentHandler.checkDoubleBooking(trigger.new, Trigger.oldMap);  
    }  
}
```

### 5.3 Trigger Handler Pattern – Separation of Concerns

#### Design Approach

To follow Salesforce best practices, the trigger logic was separated from business logic using a **Trigger Handler Class**.

- Trigger handles only event routing
- Handler class contains business logic methods

#### Benefits

- Improved code readability
- Easier maintenance and debugging
- Reusable business logic
- Better scalability for future enhancements



```

public class AppointmentHandler {

    public static void checkDoubleBooking(List<Appointment__c> newApps, Map<Id, Appointment__c> c

        Set<Id> doctorIds = new Set<Id>();

        // Collect Doctor Ids from new records
        for (Appointment__c app : newApps) {
            if (app.Doctor__c != null && app.Appointment_Date__c != null && app.Appointment_Time_

                doctorIds.add(app.Doctor__c);
            }
        }

        if (doctorIds.isEmpty()) {
            return;
        }
    }
}

```

## 5.4 SOQL Queries and Collections Usage

### SOQL Usage

SOQL (Salesforce Object Query Language) was used to fetch existing appointment records for comparison and processing.

Example use cases:

- Fetch appointments for a specific doctor
- Fetch appointments with status = Scheduled
- Fetch appointments with date < Today

### Collections Used

- **List** – To store multiple appointment records
- **Set** – To store unique doctor IDs
- **Map** – Can be used for efficient record mapping

## 5.5 Scheduled Apex – Auto Update Missed Appointments

### Business Requirement

If an appointment date has passed and the status is still “Scheduled”, it should automatically be marked as “**Missed**” to maintain accurate reporting.

### **Implementation**

A Scheduled Apex class was created with the following behavior:

- Runs once daily
- Queries appointments where:
  - Appointment Date < Today
  - Status = Scheduled
- Updates Status to “Missed”

### **Scheduling**

The class was scheduled using Salesforce Scheduler to run daily during off-peak hours.

## **5.6 Exception Handling and Error Management**

### **Implementation**

Try-catch blocks were used in update operations to handle runtime exceptions gracefully.

Key practices followed:

- Catch generic exceptions
- Log error messages using debug statements
- Prevent system crashes

## **5.7 Test Classes and Code Coverage**

### **Business Requirement**

Salesforce requires a minimum of **75% test coverage** for all Apex code before deployment.

```

public class AppointmentTestClass {

    @isTest
    static void testDoubleBookingTrigger() {

        // Create Doctor
        Doctor__c doc = new Doctor__c(
            Name = 'Test Doctor',
            Specialization__c = 'General'
        );
        insert doc;

        // Create Patient
        Patient__c pat = new Patient__c(
            Name = 'Test Patient',
            Phone_Number__c = '9999999999'
        );
        insert pat;

        // First Appointment
        Appointment__c app1 = new Appointment__c(
            Doctor__c = doc.Id,
            Patient__c = pat.Id,
            Appointment_Date__c = Date.today(),
        );
    }
}

```

## Implementation

Dedicated test classes were written to:

- Create test Patient, Doctor, and Appointment records
- Execute trigger logic
- Validate scheduling rules
- Assert expected outcomes

## Best Practices Followed

- Use of @isTest annotation

- Independent test data creation
- No dependency on existing org data
- Use of `System.assert()` for validation

## **5.8 Asynchronous Processing (Conceptual Enhancement)**

Asynchronous Apex concepts such as:

- Future Methods
- Queueable Apex
- Batch Apex

were documented for future enhancements such as:

- Bulk appointment processing
- Large data cleanup jobs
- Background notifications

# Phase 6: User Interface Development

## 6.1 UI Design Strategy

The UI design strategy was based on the following principles:

- Role-based user experience
- Minimal clicks for frequent operations
- Clear visibility of important information
- Consistent layout across objects
- Easy navigation between related records

## UI Components Implemented

- Custom Lightning App
- Custom Tabs and Navigation Bar
- Record Pages for Patient, Doctor, and Appointment
- Custom Home Page
- Utility Bar
- Optional Lightning Web Components (conceptual)

## 6.2 Custom Lightning App – Clinic CRM

### Business Requirement

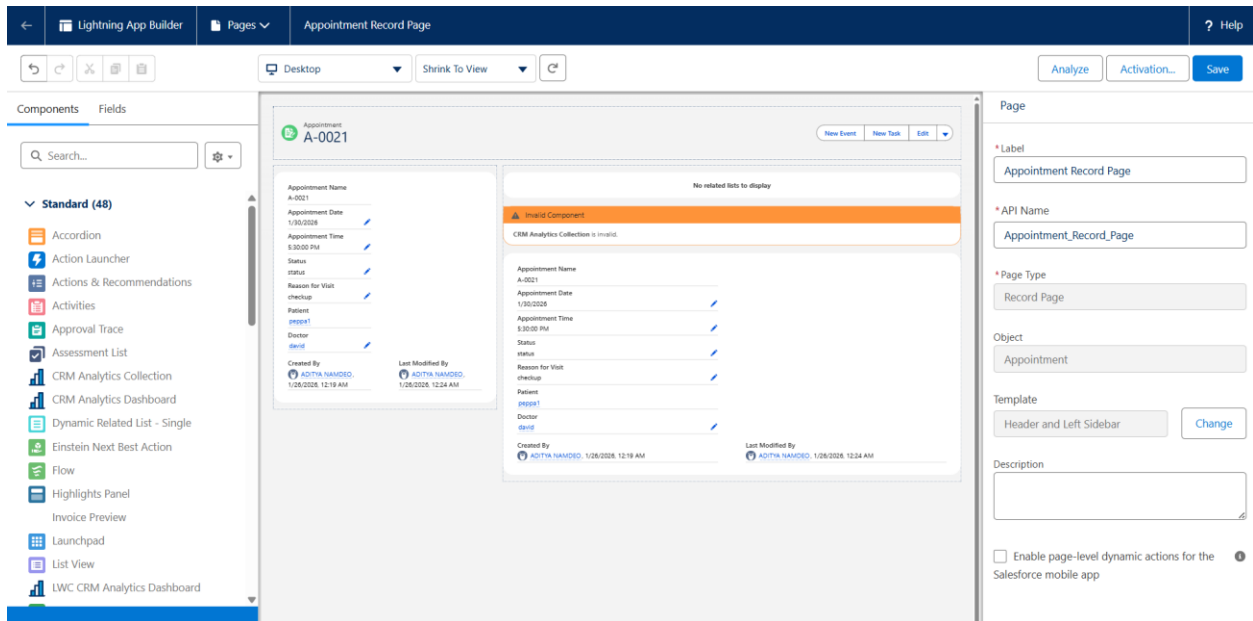
Clinic users require a dedicated workspace that shows only relevant objects and features.

### Implementation

A custom Lightning App named **Clinic CRM** was created using App Manager with the following configuration:

- Navigation Style: Standard
- Assigned Objects:

- Patient
- Doctor
- Appointment
- Reports
- Dashboards
- Assigned Profiles:
  - System Administrator
  - Standard User



## 6.3 Custom Tabs and Navigation Bar Configuration

### Business Requirement

Users should be able to quickly access frequently used objects.

### Implementation

- Custom Object Tabs were created for:
  - Patient

- Doctor
  - Appointment
- Navigation Bar was customized to include:
  - Home
  - Patients
  - Appointments
  - Doctors
  - Reports

## **6.4 Patient Record Page Design**

### **Business Requirement**

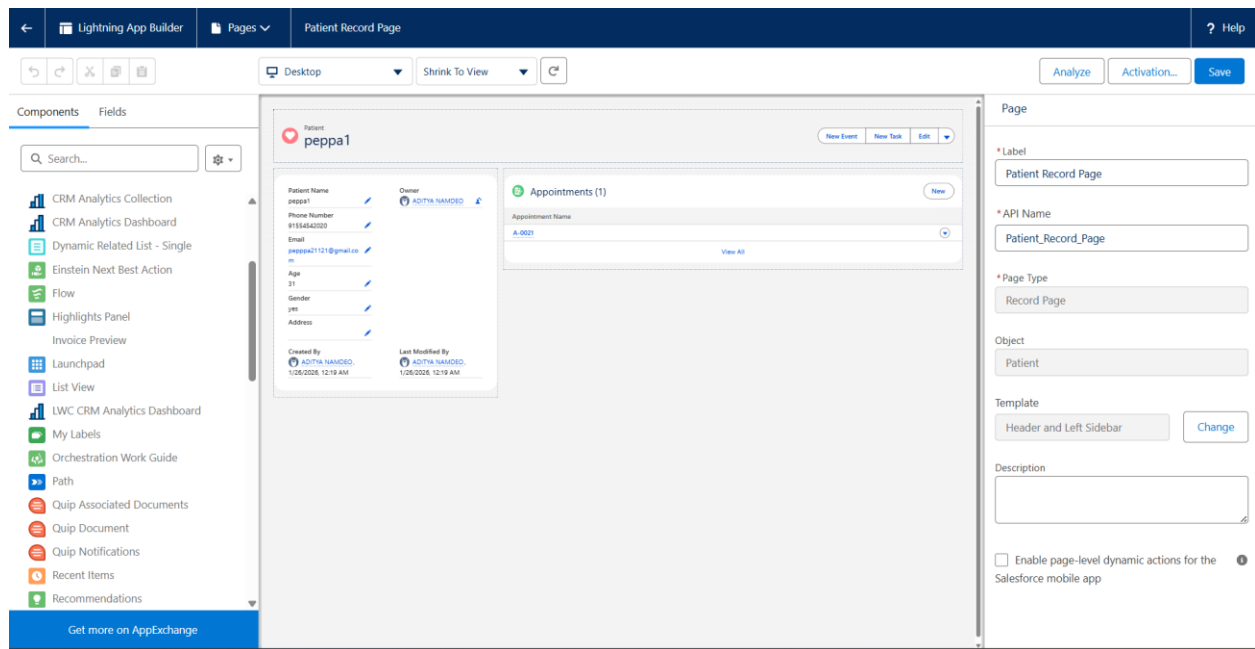
Patient details and related appointments should be clearly visible in one place.

### **Implementation**

A custom Patient Record Page was created using Lightning App Builder with the following components:

- Highlights Panel – Displays key patient information
- Record Detail – Shows all patient fields
- Related Lists – Displays related Appointments
- Tabs Component – Organizes sections logically

The page was activated as the default page for the Clinic CRM app.



## 6.5 Doctor Record Page Design

### Business Requirement

Doctors should easily view their profile details and assigned appointments.

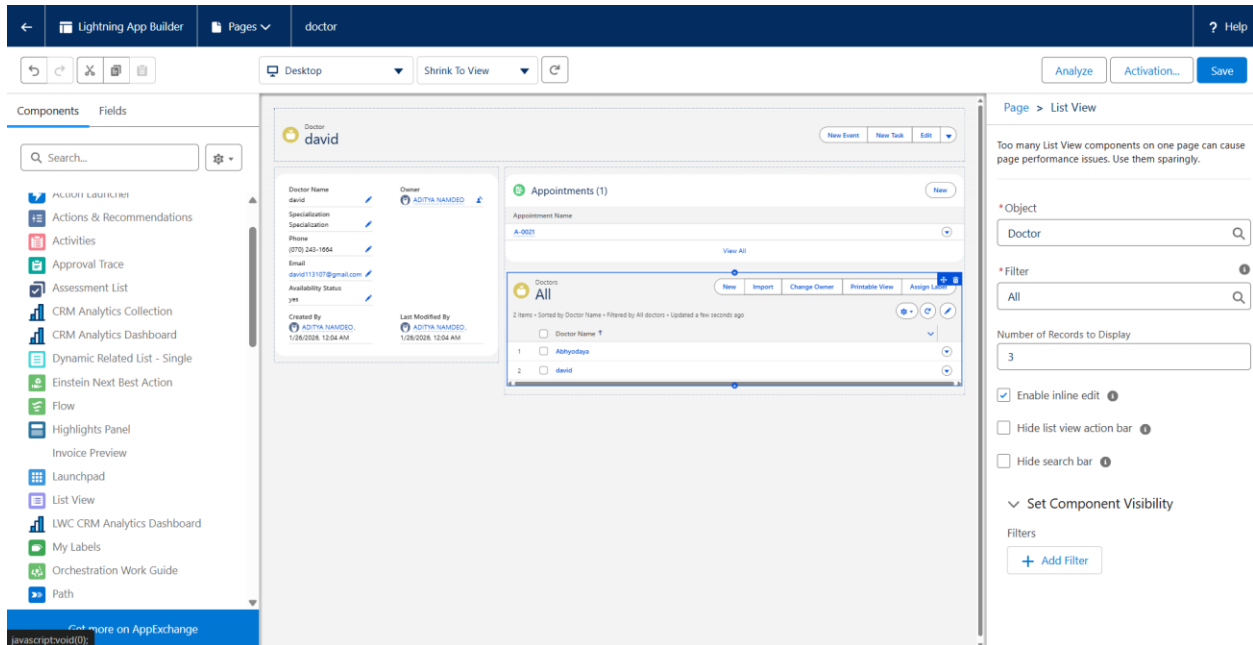
### Implementation

A custom Doctor Record Page was created with:

- Highlights Panel
- Record Detail Section
- Related Appointments List

The page was assigned specifically to the Clinic CRM app.





## 6.6 Appointment Record Page Design

### Business Requirement

The Appointment page is the most frequently used screen and must provide quick access to all relevant information.

### Implementation

A custom Appointment Record Page was created with:

- Highlights Panel showing Date, Time, Status
- Record Detail Section for booking information
- Related Patient and Doctor information

# Phase 7: Integration & External Access

## 7.1 Integration Strategy and Scope

The integration strategy for this project is designed with the following principles:

- Secure communication with external systems
- Minimal exposure of sensitive patient data
- Use of standard Salesforce integration frameworks
- Scalability for future integrations
- Compliance with authentication and authorization standards

### Integration Scenarios Considered

- Sending SMS or email reminders through external messaging services
- Integrating with payment gateways for consultation fees
- Exchanging appointment data with hospital management systems
- Synchronizing patient records with external healthcare platforms

## 7.2 Remote Site Settings – Enabling External Callouts

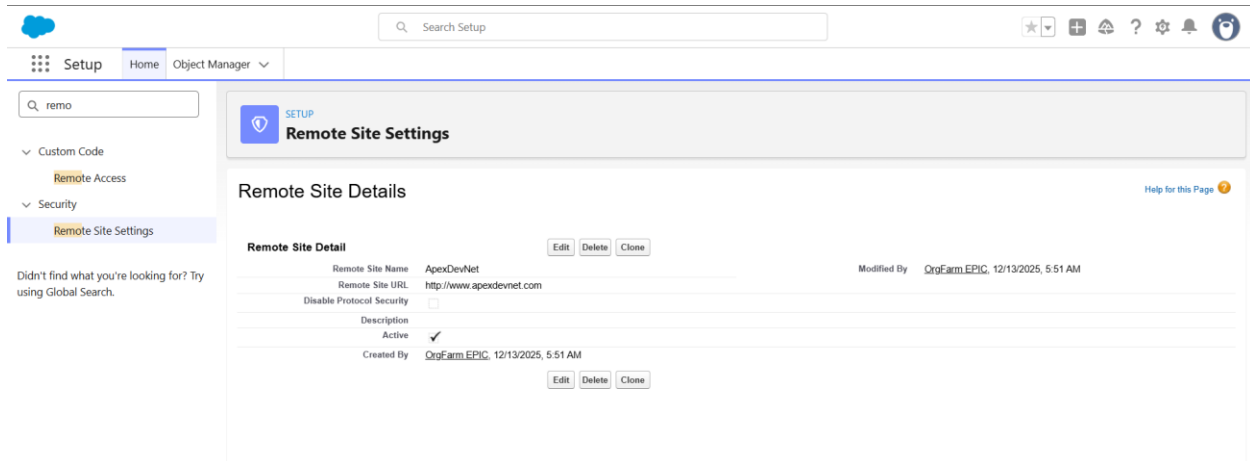
### Business Requirement

By default, Salesforce blocks all outbound HTTP callouts for security reasons. To allow communication with trusted external services, Remote Site Settings must be configured.

### Implementation

Remote Site Settings were configured with:

- Remote Site Name: Clinic\_External\_API
- Remote Site URL: Example external service endpoint
- Active: Enabled



## 7.3 Named Credentials – Secure Authentication Management

### Business Requirement

Authentication details such as usernames, passwords, and tokens should not be hardcoded in Apex code. They must be stored securely and centrally.

### Implementation

Named Credentials were created with:

- Endpoint URL of the external service
- Authentication Protocol (Password / OAuth)
- Identity Type: Named Principal

## 7.4 External Services – Low-Code API Consumption

### Conceptual Implementation

External Services allow Salesforce Flows to directly call REST APIs without writing Apex code.

### Planned Use Cases

- Call SMS API to send appointment reminders
- Call payment API to process consultation fees

## **7.5 Web Services – REST API Exposure**

### **Business Requirement**

External systems may need to fetch or update data from the clinic CRM.

### **Implementation Approach**

Salesforce REST APIs were documented to:

- Expose Appointment data
- Expose Patient basic details
- Support external system queries

### **Example Use Cases**

- Hospital system fetches today's appointments
- Reporting system pulls appointment statistics
- Mobile app retrieves patient schedules

## **7.6 Callouts – Outbound Communication to External Systems**

### **Business Requirement**

The CRM must be able to send data to external systems such as SMS gateways or notification services.

### **Implementation**

Outbound callouts were conceptually designed using:

- HttpRequest and HttpResponse classes
  - Named Credentials for endpoint resolution
  - POST requests for sending notification data
  - engagement
-

# Phase 8: Data Management & Deployment

## 8.1 Data Management Strategy

The data management strategy for this project is based on the following principles:

- Maintain clean and accurate patient data
- Prevent duplicate records
- Support bulk data loading
- Ensure data backup and recovery
- Enable controlled migration of configurations

## 8.2 Data Import Wizard – Initial Data Loading

### Business Requirement

Initial patient and doctor records must be loaded into the CRM from external sources such as Excel files.

### Implementation

The Salesforce **Data Import Wizard** was used to import:

- Patient records
- Doctor records

Key steps followed:

- Prepared CSV files with proper column mapping
- Matched CSV columns with Salesforce fields
- Validated data before import
- Executed import in batches

### **8.3 Data Loader – Advanced Data Operations**

#### **Business Requirement**

For large data volumes or advanced operations such as update, delete, and export, a more powerful tool is required.

#### **Implementation**

Salesforce **Data Loader** was documented for:

- Bulk insert of appointment data
- Bulk update of status fields
- Data export for backup purposes

### **8.4 Duplicate Management – Preventing Data Duplication**

#### **Business Requirement**

Duplicate patient records lead to inconsistent history, incorrect reporting, and poor patient experience. Duplicate prevention is critical.

#### **Implementation**

Duplicate Management was configured using:

- **Matching Rules** based on:
  - Email
  - Phone Number
- **Duplicate Rules** to:
  - Block duplicate creation
  - Alert users on potential duplicates

### **8.5 Data Export and Backup Strategy**

#### **Business Requirement**

Patient data must be backed up regularly to prevent data loss and support disaster recovery.

#### **Implementation**

The following backup strategies were defined:

- Weekly data export using Data Export Service
- Manual exports using Data Loader
- Secure storage of backup files

## **8.6 Change Sets – Controlled Deployment**

### **Business Requirement**

Configurations created in the development environment must be safely moved to production without errors.

### **Implementation**

**Change Sets** were used to deploy:

- Custom Objects
- Fields
- Validation Rules
- Flows
- Page Layouts
- Reports and Dashboards

Key practices:

- Outbound Change Set created in source org
- Components added selectively
- Inbound Change Set validated before deployment
- Deployment executed after validation

## **8.7 Managed vs Unmanaged Packages**

### **Conceptual Explanation**

- **Managed Packages:** Used for distributing commercial applications

- **Unmanaged Packages:** Used for internal deployment and learning projects

For this project, unmanaged package concepts were documented for internal distribution.

## **8.8 ANT Migration Tool and SFDX (Conceptual)**

### **ANT Migration Tool**

Used for metadata-based deployments using XML and command line.

### **Salesforce DX (SFDX)**

Used for:

- Source-driven development
- Version control integration
- CI/CD pipeline support

## **8.9 Data Security and Compliance in Data Management**

To ensure healthcare data compliance, the following practices were followed:

- Role-based data access
- Field-level security for sensitive fields
- Secure backup storage
- Controlled data exports
- Audit trail monitoring



# Phase 9: Reports, Dashboards & Security

## 9.1 Reporting and Analytics Strategy

The reporting strategy was designed with the following objectives:

- Provide real-time visibility into clinic operations
- Support daily, weekly, and monthly performance analysis
- Enable doctor-wise and date-wise monitoring
- Identify missed appointments and workload patterns
- Support management decision-making

### Types of Reports Implemented

- Tabular Reports – For simple listings
- Summary Reports – For grouped analysis
- Matrix Reports – For cross-dimensional analysis
- Joined Reports – For combining multiple datasets (conceptual)

## 9.2 Report Types Configuration

### Business Requirement

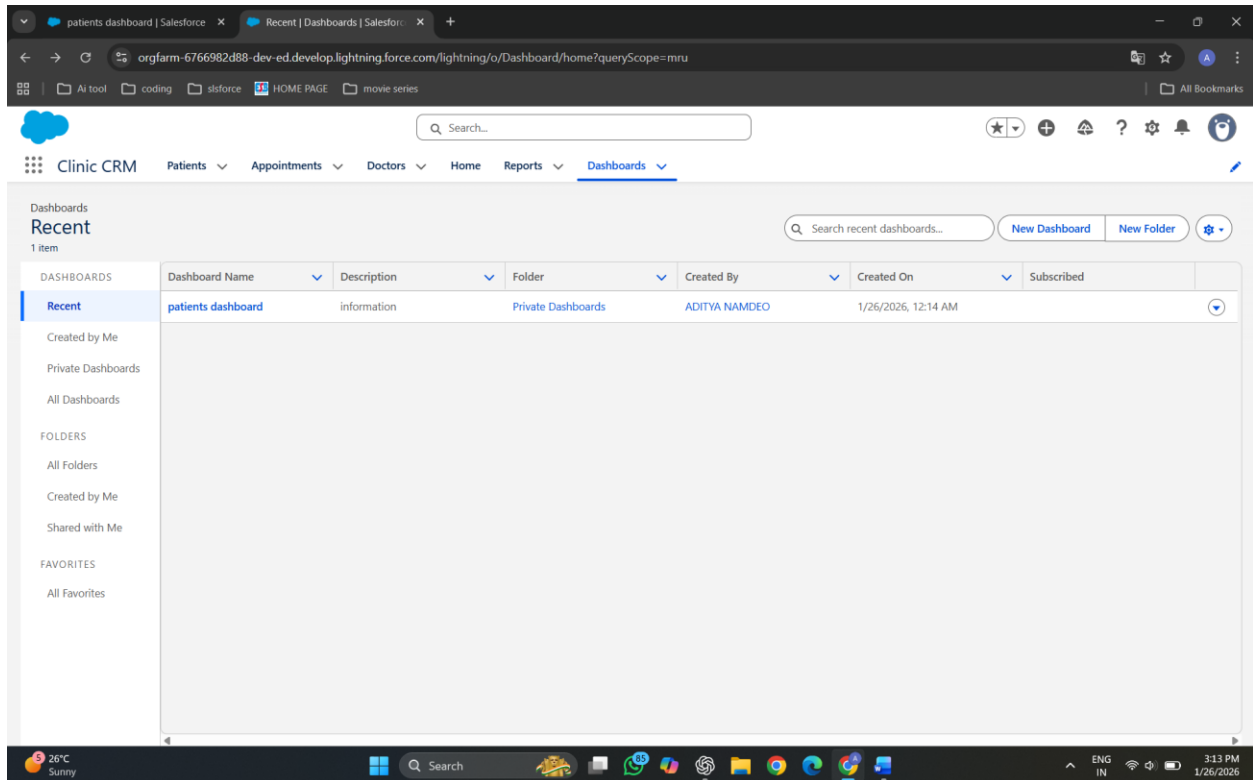
Custom objects require custom report types to enable meaningful reporting.

### Implementation

Custom Report Types were created for:

- Patient with Appointments
- Doctor with Appointments
- Appointment with Patient and Doctor

These report types enable users to analyze appointment data in relation to both patients and doctors.



## 9.3 Key Operational Reports

The following key reports were created to support clinic operations:

### 1. Appointments by Date

- Shows all appointments grouped by Appointment Date
- Used to monitor daily and weekly schedules

### 2. Appointments by Doctor

- Groups appointments by Doctor
- Shows workload distribution

### Business Value:

Helps balance doctor schedules and avoid overloading.

### 3. Missed Appointments Report

- Filters appointments with Status = Missed

- Identifies no-show patterns

**Business Value:**

Helps improve reminder strategies and patient engagement.

---

#### **4. Completed vs Cancelled Appointments**

- Summary report grouping by Status

**Business Value:**

Measures operational efficiency and cancellation trends.

---

#### **5. Patient Visit History**

- Shows appointment history per patient

### **9.4 Advanced Reporting Features**

#### **Grouping and Summaries**

- Group by Doctor, Date, and Status
- Add row-level and group-level summaries

#### **Filters and Cross-Filters**

- Filter by date ranges
- Filter by status and doctor
- Cross-filter patients with or without appointments

#### **Formulas in Reports**

- Formula fields used to calculate:
  - Total appointments per doctor
  - Percentage of missed appointments

### **9.5 Dashboards – Visual Analytics**

#### **Business Requirement**

Management and administrators require a visual summary of clinic performance.

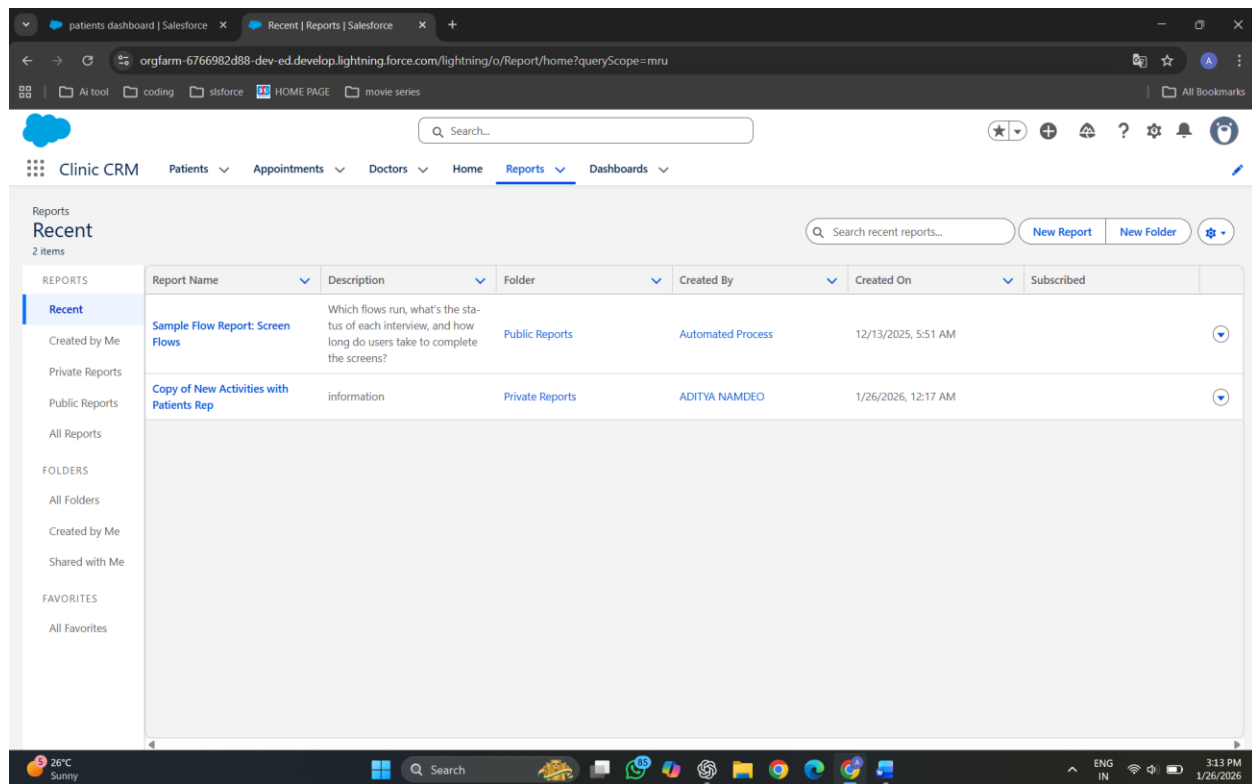
## Implementation

A custom dashboard named **Clinic Operations Dashboard** was created with the following components:

- Today's Appointments (Bar Chart)
- Appointments by Status (Donut Chart)
- Doctor-wise Workload (Horizontal Bar)
- Missed Appointments Trend (Line Chart)
- Total Patients Count (Metric)

## Dynamic Dashboards

Dynamic dashboards were configured to display data based on the logged-in user's access level.



## 9.6 Security Strategy and Principles

The security strategy for this project was designed based on:

- Principle of least privilege
- Role-based access control
- Protection of sensitive patient data
- Compliance with healthcare data standards
- Auditability and traceability

### **Security Layers Implemented**

- Profiles
- Roles
- Permission Sets
- Organization-Wide Defaults (OWD)
- Sharing Rules
- Field-Level Security
- Session and Login Policies

# Phase 10: Quality Assurance Testing.

---

## 10.1 Testing Strategy and Objectives

The testing strategy for this project was designed with the following objectives:

- Verify functional correctness of all features
- Validate automation flows and business rules
- Ensure data integrity and security controls
- Confirm role-based access behavior
- Detect defects early and ensure system stability

## Types of Testing Performed

- Functional Testing
- Automation Testing
- Integration Readiness Testing
- Security Testing
- User Acceptance Testing (UAT)
- Increases user confidence
- Ensures compliance with quality standards

---

## 10.2 Test Planning and Test Case Design

### Test Planning

A formal test plan was prepared covering:

- Scope of testing
- Features to be tested
- Test environment (Developer Org)
- Test data preparation

- Roles involved in testing

### **Test Case Design Principles**

Each test case included:

- Test Case ID
- Test Scenario / Use Case
- Test Steps and Input Data
- Expected Result
- Actual Result
- Pass / Fail Status
- Supporting Screenshots

## **10.3 Functional Testing of Core Features**

### **1. Patient Record Management**

#### **Test Scenarios:**

- Create new Patient record
- Update patient contact details
- View patient history

#### **Expected Results:**

- Patient record saved successfully
- All fields stored correctly
- Record visible to authorized users

#### **Outcome:**

All patient record operations performed as expected.

---

### **2. Doctor Record Management**

#### **Test Scenarios:**

- Create Doctor profile
- Assign specialization
- Update availability status

**Expected Results:**

- Doctor record created correctly
- Fields saved and displayed accurately

**Outcome:**

Doctor management functionality validated successfully.

---

### **3. Appointment Booking**

**Test Scenarios:**

- Book new appointment
- Assign patient and doctor
- Save appointment with valid date and time

**Expected Results:**

- Appointment created successfully
- Status auto-set to “Scheduled”
- Validation rules enforced

**Outcome:**

Appointment creation and validation rules worked correctly.

---

## **10.4 Automation Testing**

### **1. Validation Rule Testing**

**Scenario:** Try to book appointment with past date

- Expected Result: System should block save
- Actual Result: Error message displayed



- Status: Passed

## **2. Record-Triggered Flow Testing**

**Scenario:** Create new appointment

- Expected Result: Status auto-set to “Scheduled”
- Actual Result: Status updated automatically
- Status: Passed

## **3. Email Alert Testing**

**Scenario:** Book new appointment

- Expected Result: Confirmation email sent to patient
- Actual Result: Email received successfully
- Status: Passed

## **4. Scheduled Flow Testing**

**Scenario:** Appointment scheduled for tomorrow

- Expected Result: Reminder email sent one day before
- Actual Result: Reminder email delivered
- Status: Passed

## **5. Task Automation Testing**

**Scenario:** Mark appointment as Completed

- Expected Result: Follow-up task created
- Actual Result: Task created successfully
- Status: Passed

---

## **10.5 Integration Readiness Testing**

Although no live external integration was implemented, integration readiness was tested by:

- Verifying Remote Site Settings configuration
- Validating Named Credentials setup

- Testing sample callout logic in sandbox
- Verifying API access using Workbench

## **10.6 Security and Access Control Testing**

### **Profiles and Role Testing**

#### **Scenarios Tested:**

- Receptionist access to create and edit records
- Doctor access to view only assigned appointments
- Admin access to all objects

#### **Expected Results:**

- Role-based access enforced correctly
- Unauthorized actions blocked

#### **Outcome:**

All access control rules functioned as expected.

---

### **Field-Level Security Testing**

#### **Scenarios Tested:**

- Doctor attempts to edit patient email
- Receptionist edits patient contact details

#### **Expected Results:**

- Doctor blocked from editing sensitive fields
- Receptionist allowed to edit permitted fields

#### **Outcome:**

Field-level security validated successfully.